INSTRUCTION MANUAL • MVP VERTICAL SLICE

Whetstone

The post-build skill layer for people who direct coding agents

---

Whetstone watches what you delegate to a coding agent, teaches you the single sharpest directing principle you are missing, and then *verifies* whether you reuse it, unaided, in a later build. It runs on your machine, interrupts rarely, and keeps no account and no telemetry. This manual covers the one practice the MVP ships today: **structured-output boundary validation** — requiring model output to be schema-validated and rejected before it crosses a state boundary.

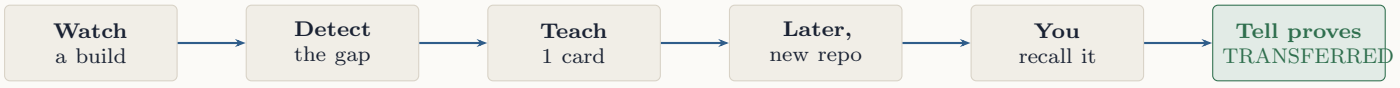
> **What state this is**
>
> Engine complete and proven: 8 phases, 100+ tests, an adversarial schema gate, all on `main`. Every phase carries a machine-checkable `phase_verified` record — "done" is evidence, not a claim. A live `Stop`-hook orchestrator makes it installable today.

What it does for you

You increasingly direct an agent instead of writing the code yourself. That builds the agent's capability and quietly erodes yours. Whetstone is the counterweight: it turns your building hours back into *your* compounding judgement.

- **Watches** each completed task (via a Claude Code `Stop` hook) and notices when you shipped model output across a boundary with no validation.
- **Teaches** one principle, once, at command altitude — three lines (Direction / Principle / Tell), never a code dump.
- **Waits**, then when a comparable situation appears in a *different* repo, prompts you to recall the principle in your own words.
- **Verifies** transfer from an *artifact*, not a guess: it only records "transferred" when a discriminating test proves the malformed output can no longer reach the sink. This is the whole point — the claim is earned, never inferred.

The loop, at a glance



Steps 1–5 are deterministic and run with no model in the loop. The green step is the only strong claim, and it is gated by an executable test, not an opinion.

Install

Prerequisites: Python 3, `git`, and `pip install jsonschema pytest`.

1. Clone the repo and record its path. Clone anywhere; everything below is relative to your clone (shown as `$WHETSTONE`).

```
git clone https://github.com/RoryPeterRoberts/Whetstone.git
cd Whetstone
export WHETSTONE="$(pwd)"     # absolute path to your clone
```

2. Verify the engine on your machine (from the repo root):

```
python3 -m pytest whetstone -q
python3 spec/schema_tests/run_schema_tests.py
```

You should see every test pass and `OK: schemas strict-compile; 9 valid accepted; 12 invalid rejected`.

3. Register the Stop hook in `~/.claude/settings.json` so Claude Code runs Whetstone after each completed task. Claude Code needs an *absolute* path, so substitute the value of `$WHETSTONE` from step 1 (e.g. `/home/you/code/Whetstone`):

```
{
  "hooks": {
    "Stop": [
      { "hooks": [
        { "type": "command",
          "command": "python3 $WHETSTONE/whetstone/whetstone_hook.py" }
      ] }
    ]
  }
}
```

4. Turn capture on:

```
cd $WHETSTONE/whetstone
python3 cli.py on
```

> **The hook can never break your session**
>
> `whetstone_hook.py` fails open: malformed input or any internal error is swallowed and it exits 0. It reads the hook JSON on stdin, records to a local ledger, and writes diagnostics only to stderr. It never blocks or edits your work.

The commands

Run these from the `whetstone/` directory. Status is a command, not a dashboard.

Command	What it does
`cli.py on` / `off`	Enable or disable capture.
`cli.py status`	Command-altitude learner state from the ledger: how many principles you have ever recalled and transferred, and each repo's disposition.
`cli.py metrics`	The pilot metrics (see next page), as JSON.
`cli.py diagnostic`	Versions, capability detection, and the trust contract.
`cli.py delete-data`	Remove your local product records (events, direction records, projections) on demand. Leaves the governance ledger and legacy inputs alone.

`status` looks like this:

```
capture: on
```

```
principles ever transferred: 1
principles ever recalled:    1
repositories seen:           2
  r_9f3c1a...  p_structured_output_boundary  transferred
```

Reading the metrics

`cli.py metrics` computes everything from the append-only event ledger — nothing is tracked separately, so the numbers can always be re-derived and never drift.

Metric	Meaning
`acceptance_rate`	Of findings surfaced, the fraction you accepted.
`top_finding_precision`	Of findings you actually judged, the fraction that were real.
`false_positive_rate`	Findings surfaced that were not a real actionable gap.
`application_rate`	Of accepted findings, the fraction you applied.
`recurrence_rate`	Principles that recur across more than one repo.
`prompted_recall_rate`	Of recalls prompted, the fraction you answered with a matching direction.
`unaided_transfer_count`	**The north-star.** Times you reused a principle, unaided, in a new repo, proven by a discriminating tell.
`unaided_transfer_rate`	Transfers over lessons that had a genuinely *comparable later opportunity* — not total lessons.

> **Why the denominator matters**
>
> The transfer rate divides by the number of lessons for which a comparable later opportunity actually occurred, not by every lesson you were ever shown. A tool that divided by total lessons would look worse the more it taught, and could be gamed by never following up. This denominator is the honest one.

The trust contract

- **No account, no telemetry, no cloud.** Everything is local files in the repo.
- **Your raw transcript never leaves the machine** and is never copied. Only a redacted direction excerpt is stored, and a deterministic redaction pass strips secrets *before* anything is written.
- **No model sees your direction text** in this slice — the matcher is deterministic and local.
- **Nothing is enforced.** Whetstone records risk as metadata; it never blocks an action or edits your code. Applying any lesson is your choice.
- **`delete-data` means it.** One command removes your captured records.

What is proven, and what is not

> **Proven end to end**
>
> The full path — watch, detect, capture your direction, match it, run a discriminating tell, and record `transferred` only across different repositories — passes as an acceptance

test, driven by a *real* executable tell (not a mock). Every "must-not-transfer" case (agent inserted it, irrelevant direction, a vacuous test, a changed worktree) is rejected structurally.

The honest edge

The artifact tell is proven against a registered discriminating test for the structured-output practice. Whetstone does *not* yet auto-generate a discriminating test for arbitrary code — in the wild, the transfer leg needs that test to exist. The live hook today does the deterministic work reliably (trigger, repo identity, direction capture, detection, recording); full automated transfer verification on any repo is the next slice, not a present claim. The MVP also watches for exactly *one* practice by design.

Verify it yourself

You never have to take the tool's word for anything.

```
# re-run the whole suite and the adversarial schema gate (from the repo root):
python3 -m pytest whetstone -q
python3 spec/schema_tests/run_schema_tests.py
```

The schema gate compiles every schema and proves it *rejects* 12 hand-built invalid states while accepting 9 valid ones — the JSON files enforce the rules, they do not merely describe them. The per-phase evidence ledger (`build_evidence.jsonl`) is written locally by `spec/record_phase.py` as each phase is built; like `runs.jsonl` it is git-ignored (local runtime state), so a fresh clone reproduces the proof by re-running the two commands above rather than trusting a checked-in file.

Whetstone MVP vertical slice • practice `p_structured_output_boundary` • manual built 2026-07-12. Companions: *whetstone_strongest_version.pdf* (thesis) and *whetstone_implementation.pdf* (engine baseline).